TeSpec

Public interface and evidence artifacts

A new problem runs directly from its source package and concrete bindings—no case registration is required.

INPUT PACKAGE

One test problem

Self-contained source and specification inputs

Annotated C source
implementation + QCP spec

Specification dependencies
headers · Coq .v · strategies
included only when referenced

Concrete test data
binds.json
or a reusable job.json

COMMAND-LINE INTERFACE

python3 -m spectest

analyze — **SOURCE --function F --write-binds B**
Discover required args, With variables, and optional type instances.

run — **SOURCE --function F --binds B [-I DIR]**
Directly execute a new problem and check every concrete binding.

check — **job.json**
Run the same interface from a portable, reusable job description.

check-proof — **vc/manifest.json**
Validate an explicitly manual residual proof with the Coq kernel.

AUDITABLE OUTPUT

Per-bind evidence

Stable files for inspection and reproduction

Binding template
binds.json

Result report
report.json · per-bind status
PASS · FAIL · UNKNOWN · ERROR

Execution evidence
specialized.c · stdout · stderr

Verification-condition bundle
goal · auto · manual · manifest

RESULT CONTRACT — PASS · FAIL · UNKNOWN · ERROR — Each status is reported independently for every concrete bind.

Figure 2. TeSpec exposes four commands and preserves all generated evidence in a reproducible, per-binding artifact directory.